Parul® University
Vadodara, Gujarat

NAAC GRADE A++

Information and Communication Technology

Cryptography
Unit-4

Debojyoti Chakraborty
Department of CSE – Cyber Security,
PIET (Parul University)
Techdefence labs

Content

- Digital Signature
- Digital Signature properties
- Requirements and security
- Various digital signature schemes (Elgamal and Schnorr)
- Digital Signature Algorithm / Digital Signature Standard

Cryptographic goals

Confidentiality

Symmetric-key ciphers:

- Block ciphers
- Stream ciphers

Public-key ciphers

Data integrity

Arbitrary length hash functions

Message Authentication codes (MACs)

Digital signatures

Authentication

Entity authentication

Authentication primitives

Message authentication

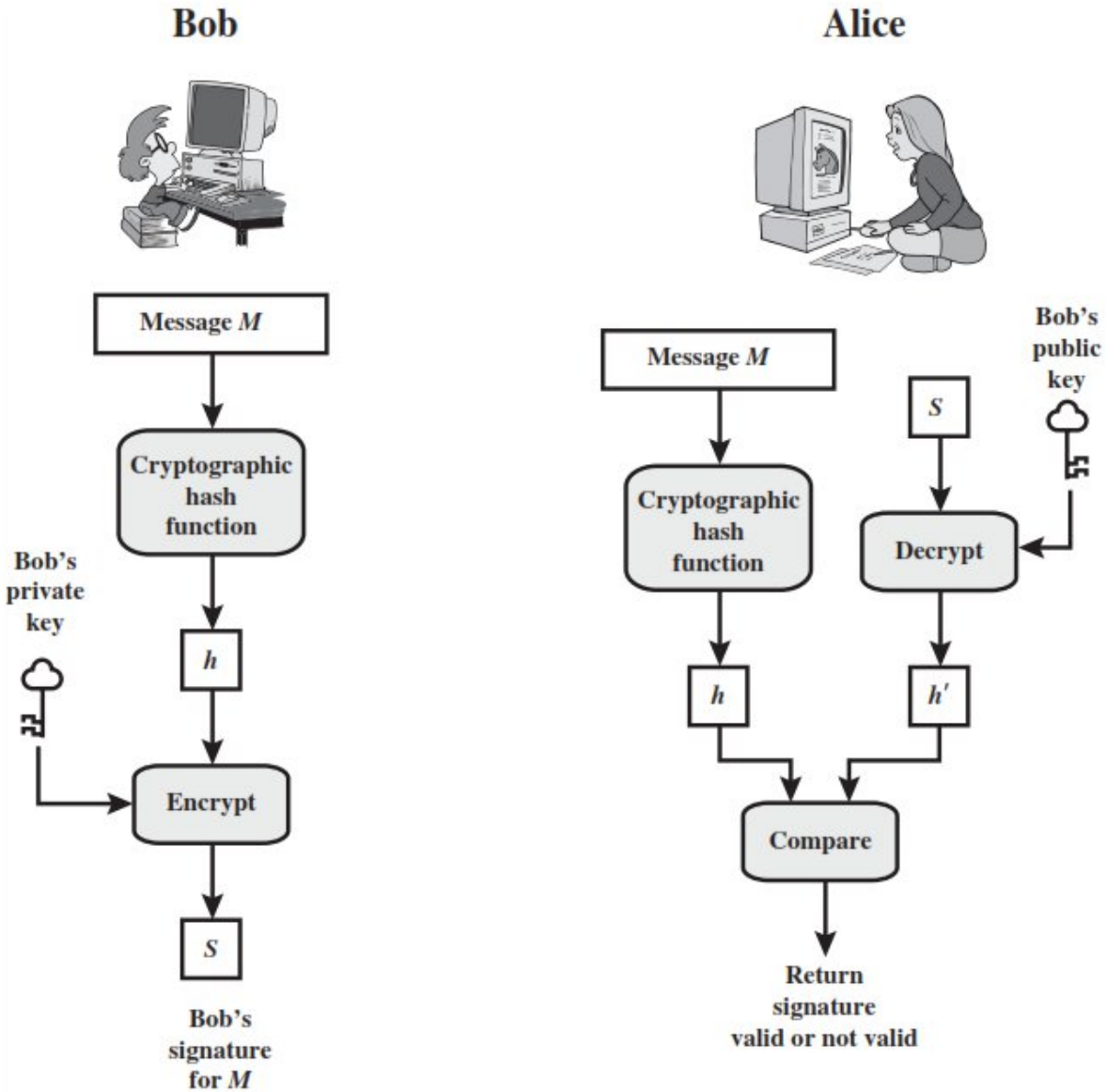
- MACs
- Digital signatures

Non-repudiation

Digital signatures

Digital Signature

- A **digital signature** is an authentication mechanism that enables the creator of a message to attach a code that acts as a **signature**.
- Typically the **signature** is formed by taking the hash of the message and encrypting the message with the creator's private key.
- The **signature** guarantees the source and integrity of the message.
- The **digital signature standard (DSS)** is an NIST standard that uses the secure hash algorithm (SHA).


Bob
Alice
Message M
Cryptographic hash function
Bob's private key
h
Encrypt
S
Bob's signature for M
Message M
Bob's public key
S
Cryptographic hash function
Decrypt
h
h'
Compare
Return signature valid or not valid

Hash code, MAC and Digital Signature

Hash Code

- A **hash** of the message, if appended to the message itself, only protects against accidental changes to the message, as an attacker who modifies the message can simply calculate a new hash and use it instead of the original one. So this **only gives integrity**.

MAC

- A message authentication code (MAC) (sometimes also known as keyed hash) protects against message forgery by anyone who doesn't know the secret.
- This means that the receiver can forge any message – thus we have both **integrity** and **authentication** (as long as the receiver doesn't have a split personality), **but not non-repudiation**.

Hash code, MAC and Digital Signature

Digital Signature

- A **digital signature** is created with a private key, and verified with the corresponding public key of an asymmetric key-pair.
- Only the holder of the private key can create this signature, and normally anyone knowing the public key can verify it.

Attacks and Forgeries

- **Key-only attack:** C only knows A's public key.
- **Known message attack:** C is given access to a set of messages and their signatures.
- **Generic chosen message attack:** C chooses a list of messages before attempting to breaks A's signature scheme, independent of A's public key. C then obtains from A valid signatures for the chosen messages. The attack is generic, because it does not depend on A's public key; the same attack is used against everyone.
- **Directed chosen message attack:** Similar to the generic attack, except that the list of messages to be signed is chosen after C knows A's public key but before any signatures are seen.
- **Adaptive chosen message attack:** C is allowed to use A as an "oracle." This means the A may request signatures of messages that depend on previously obtained message–signature pairs.

Attacks and Forgeries

- **Total break:** C determines A's private key.
- **Universal forgery:** C finds an efficient signing algorithm that provides an equivalent way of constructing signatures on arbitrary messages.
- **Selective forgery:** C forges a signature for a particular message chosen by C.
- **Existential forgery:** C forges a signature for at least one message. C has no control over the message. Consequently, this forgery may only be a minor nuisance to A.

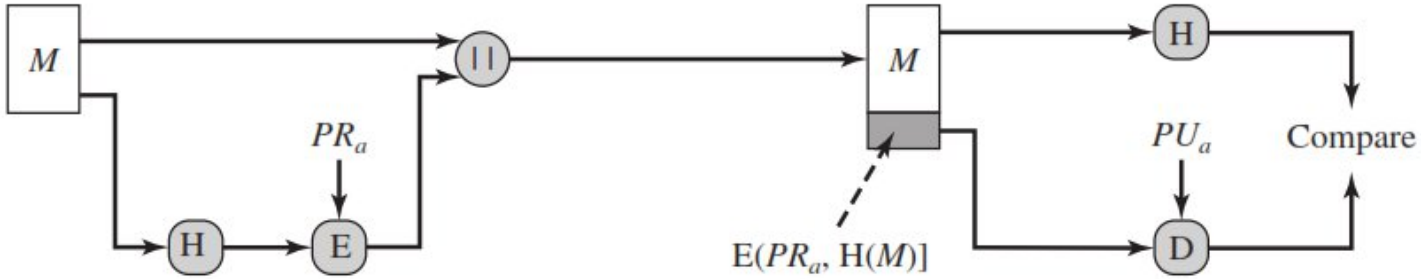
Digital Signature Requirements

1. The signature must be a **bit pattern** that depends on the message being signed.
2. The signature must use some information **unique** to the sender to prevent both forgery and denial.
3. It must be relatively **easy to produce** the digital signature.
4. It must be relatively **easy to recognize** and **verify** the digital signature.
5. It must be computationally **infeasible to forge** a digital signature, either by constructing a new message for an existing digital signature or by constructing a fraudulent digital signature for a given message.
6. It must be **practical to retain a copy** of the digital signature in storage.

- The **DSS** uses an algorithm that is designed to provide only the digital signature function.
- Unlike RSA, it cannot be used for encryption or key exchange.

RSA Approach

- In the **RSA** approach, the message to be signed is input to a hash function that produces a **secure hash code** of fixed length.
- This hash code is then encrypted using the sender's private key to form the **signature**.
- Both the message and the signature are then transmitted.
- The recipient takes the message and produces a **hash code**.

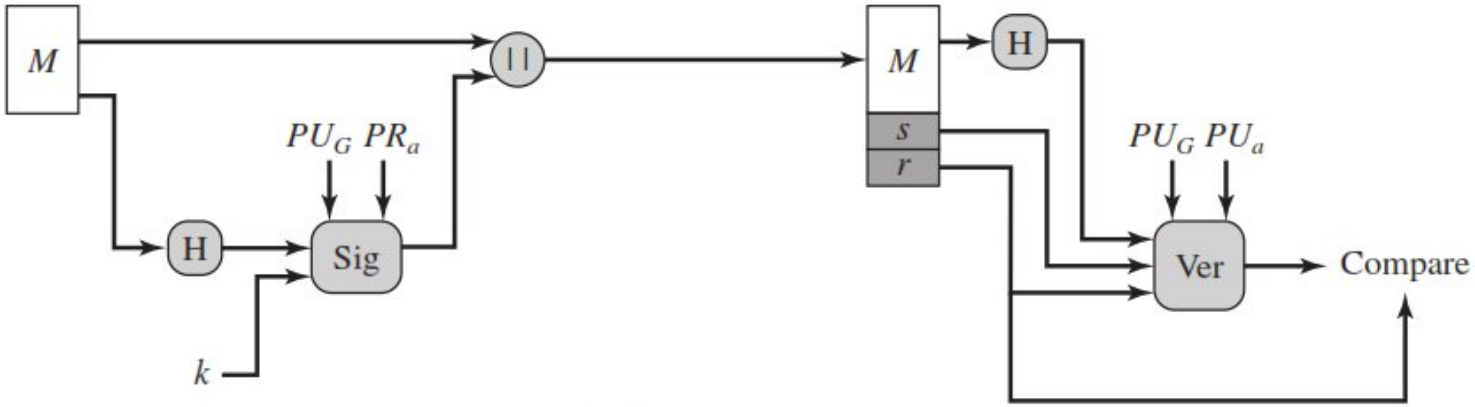


(a) RSA approach

- The recipient also decrypts the signature using the sender's public key.
- If the calculated hash code matches the decrypted signature, the signature is accepted as valid.
- Because only the sender knows the private key, only the sender could have produced a valid signature.

DSA Approach

- The **hash code** is provided as input to a **signature function** along with a random number ***k*** generated for this particular signature.
- The signature function also depends on the sender's private key **(*PRa*)** and a set of parameters known to a group of communicating principals.
- We can consider this set to constitute a global public key **(*PU*)**
- The result is a signature consisting of two components, labelled ***s*** and ***r***.



(b) DSA approach

- At the receiving end, the hash code of the incoming message is generated.
- This plus the signature is input to a **verification function**.
- The verification function also depends on the global public key as well as the sender's public key **(*PUa*)**, which is paired with the sender's private key.
- The output of the verification function is a value that is equal to the signature component ***r*** *if the signature is valid*.
- *The signature* function is such that only the sender, with knowledge of the private key, could have produced the valid signature.

Digital Signature Algorithm

Global Public-Key Components

p prime number where $2^{L-1} < p < 2^L$
for $512 \leq L \leq 1024$ and L a multiple of 64;
i.e., bit length of between 512 and 1024 bits
in increments of 64 bits

q prime divisor of $(p - 1)$, where $2^{N-1} < q < 2^N$
i.e., bit length of N bits

g $= h(p - 1)/q \bmod p$,
where h is any integer with $1 < h < (p - 1)$
such that $h^{(p-1)/q} \bmod p > 1$

User's Private Key

x random or pseudorandom integer with $0 < x < q$

User's Public Key

$y = g^x \bmod p$

User's Per-Message Secret Number

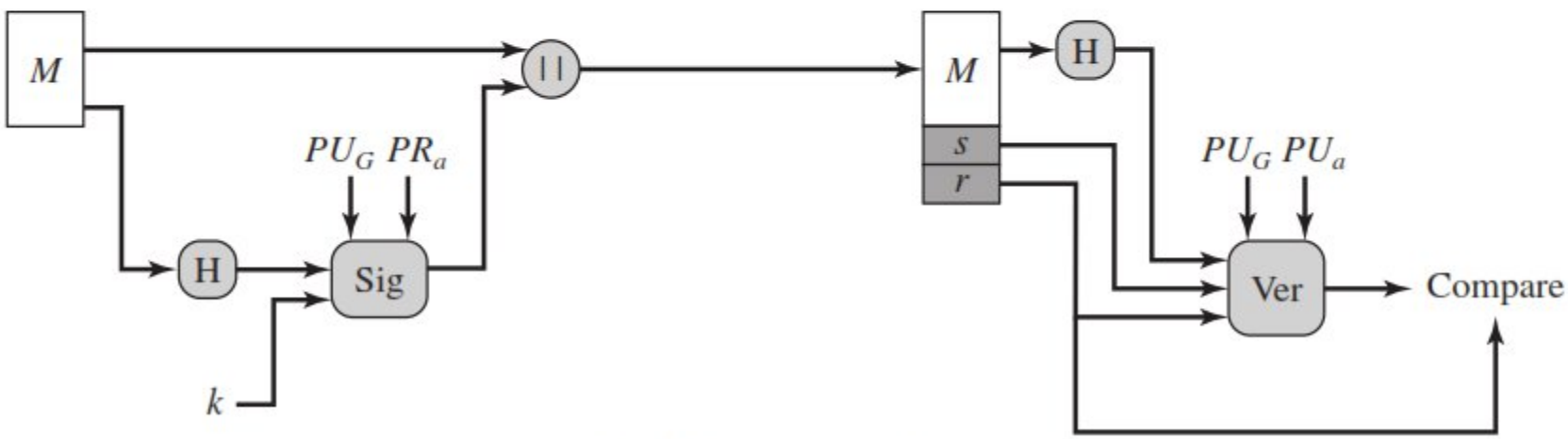
k random or pseudorandom integer with $0 < k < q$

Signing

$$r = (g^k \bmod p) \bmod q$$

$$s = [k^{-1}(H(M) + xr)] \bmod q$$

$$\text{Signature} = (r, s)$$



Verifying

$$w = (s')^{-1} \bmod q$$

$$u_1 = [H(M')w] \bmod q$$

$$u_2 = (r')w \bmod q$$

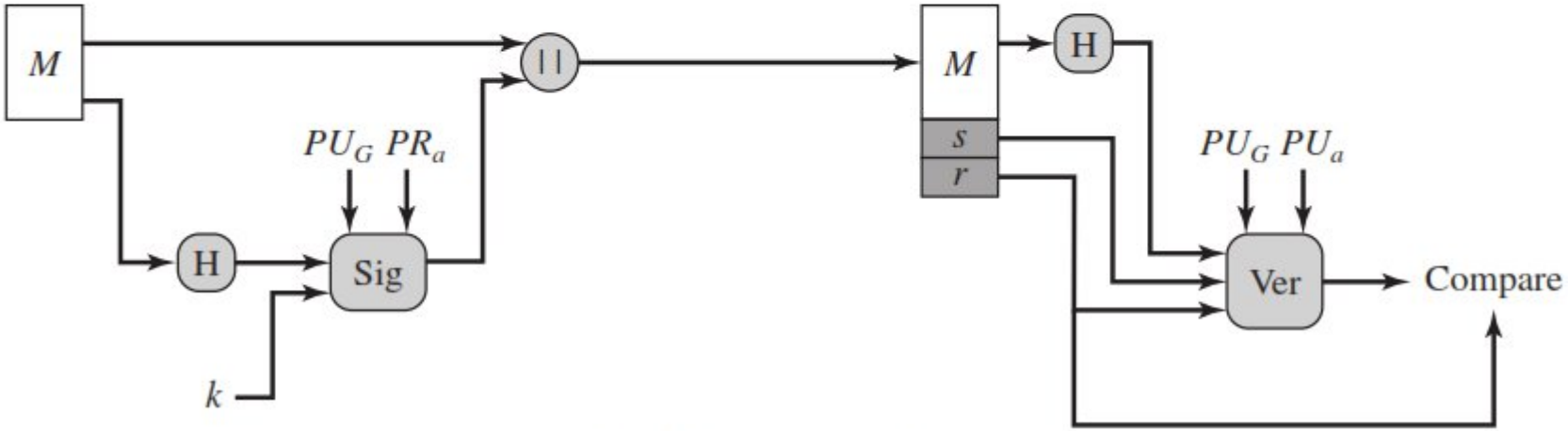
$$v = [(g^{u1}\, y^{u2}) \bmod p] \bmod q$$

TEST: $v = r'$

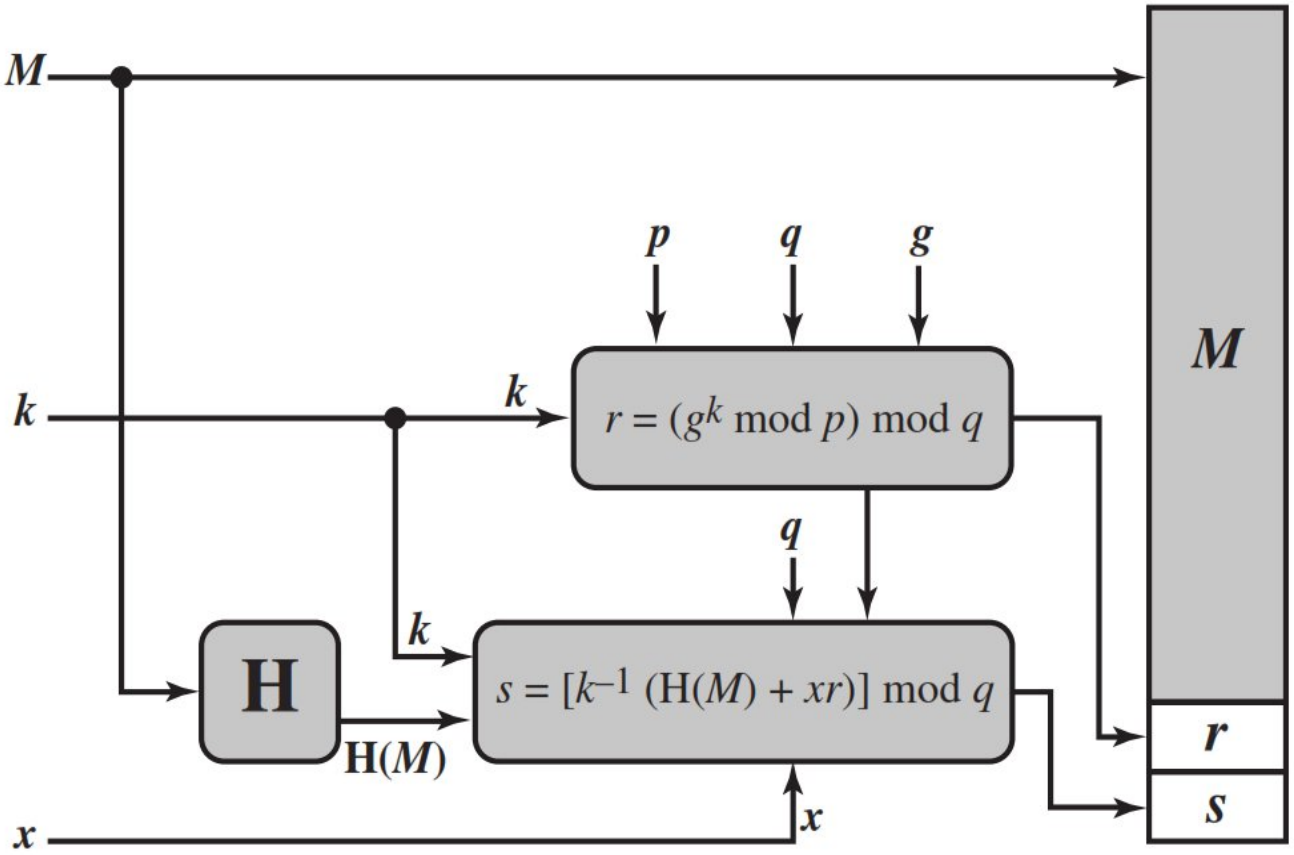
M = message to be signed

$H(M)$ = hash of M using SHA-1

M', r', s' = received versions of M, r, s

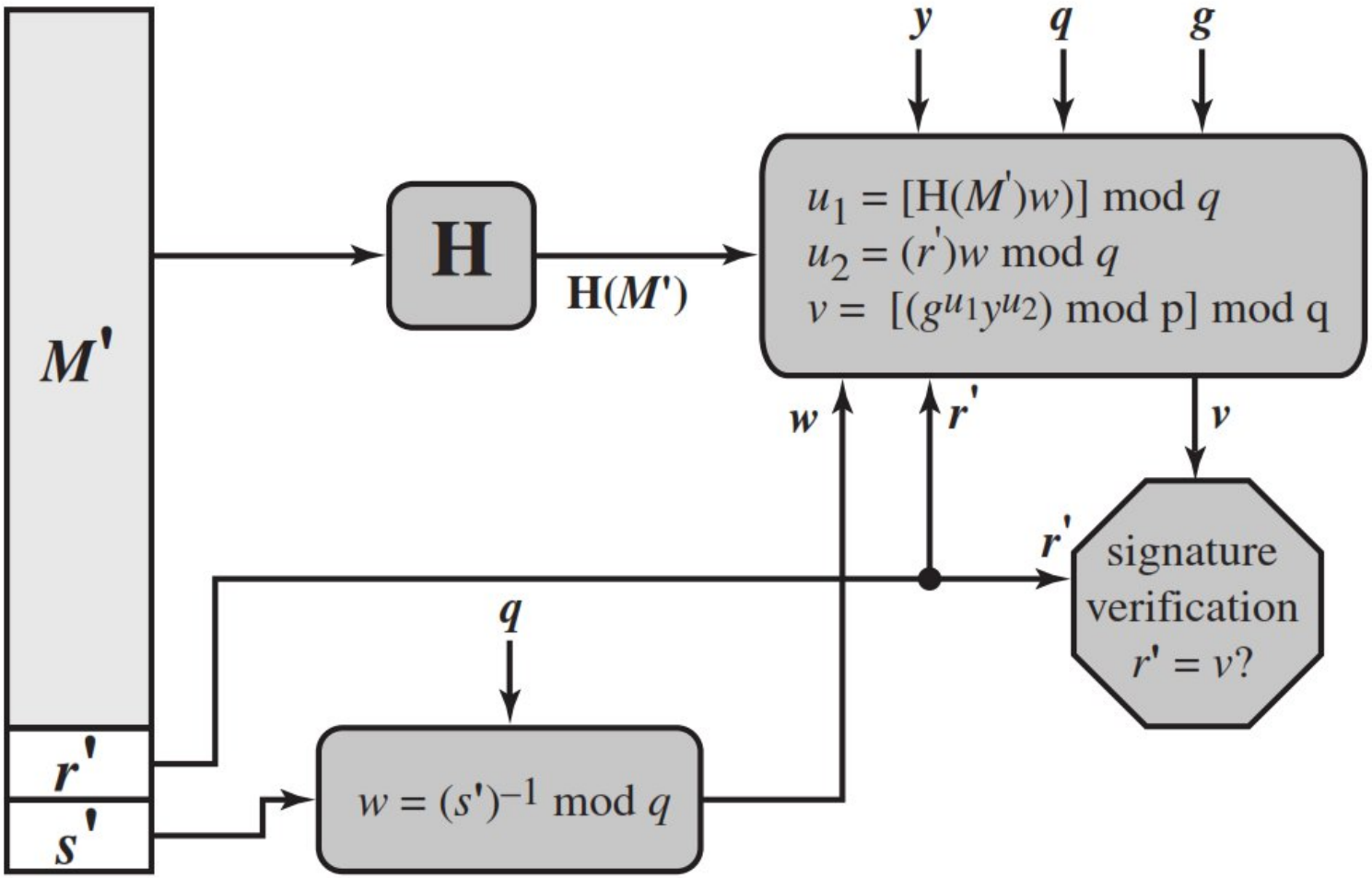


DSA Signing



(a) Signing

DSA Verifying



ElGamal Digital Signatures

- Uses private key for encryption (signing)
- Uses public key for decryption (verification)
- Each user (eg. A) generates their key
  - chooses a secret key (number): $1 < x_A < q-1$
  - compute their **public key**: $y_A = a^{xA} \bmod q$

ElGamal Digital Signature

- Alice signs a message M to Bob by computing
  - the hash $m = H(M)$, $0 <= m <= (q-1)$
  - chose random integer K with $1 <= K <= (q-1)$ and $\gcd(K, q-1)=1$
  - compute temporary key: $S_1 = a^k \bmod q$
  - compute K^{-1} the inverse of $K \bmod (q-1)$
  - compute the value: $S_2 = K^{-1}(m - x_A S_1) \bmod (q-1)$
  - signature is: (S_1, S_2)
- Any user B can verify the signature by computing
  - $V_1 = a^m \bmod q$
  - $V_2 = y_A^{S1} \; S_1^{S2} \bmod q$
  - Signature is valid if $V_1 = V_2$

ElGamal Signature Example

- Use field GF(19) $q=19$ and $a=10$
- Alice computes her key:
  - A chooses $x_A=16$ & computes $y_A=10^{16} \bmod 19 = 4$
- Alice signs message with hash $m=14$ as $(3,4)$:
  - choosing random $K=5$ which has $\gcd(18,5)=1$
  - computing $S_1 = 10^5 \bmod 19 = 3$
  - finding $K^{-1} \bmod (q-1) = 5^{-1} \bmod 18 = 11$
  - computing $S_2 = 11(14-16.3) \bmod 18 = 4$
- Any user B can verify the signature by computing
  - $V_1 = 10^{14} \bmod 19 = 16$
  - $V_2 = 4^3.3^4 = 5184 = 16 \bmod 19$
  - since $16 = 16$ signature is valid

Outline

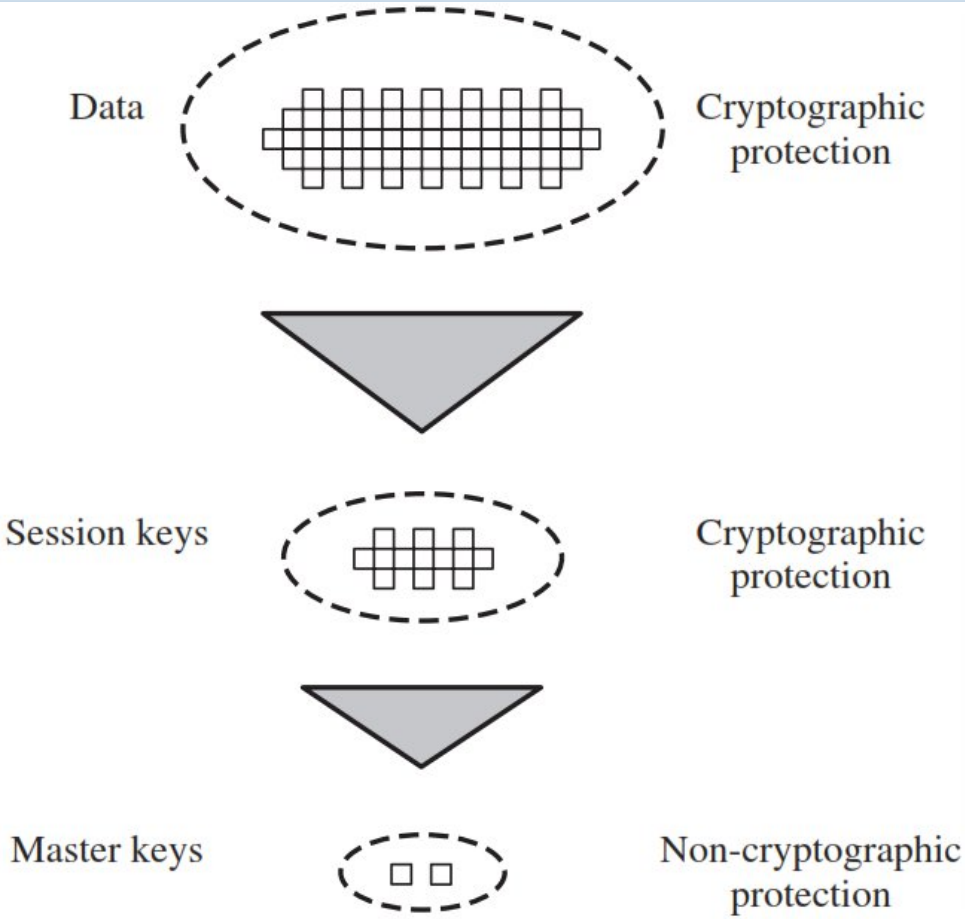
- Key management and distribution
- Symmetric key distribution using symmetric encryption
- Symmetric key distribution asymmetric encryption
- Distribution of public keys
- X.509 certificates
- Public key infrastructure (PKI)

Key Distribution

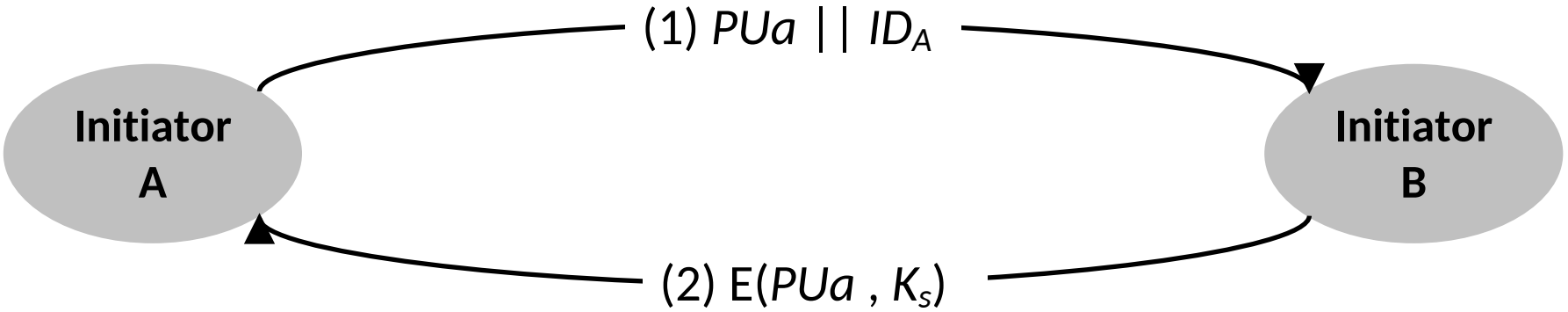
- **Key distribution** is the function that delivers a key to two parties who wish to exchange secure encrypted data.
- Some sort of mechanism or protocol is needed to provide for the secure distribution of keys.
- Key distribution often involves the use of **master keys**, which are infrequently used and are long lasting, and **session keys**, which are generated and distributed for temporary use between two parties.

Key Hierarchy

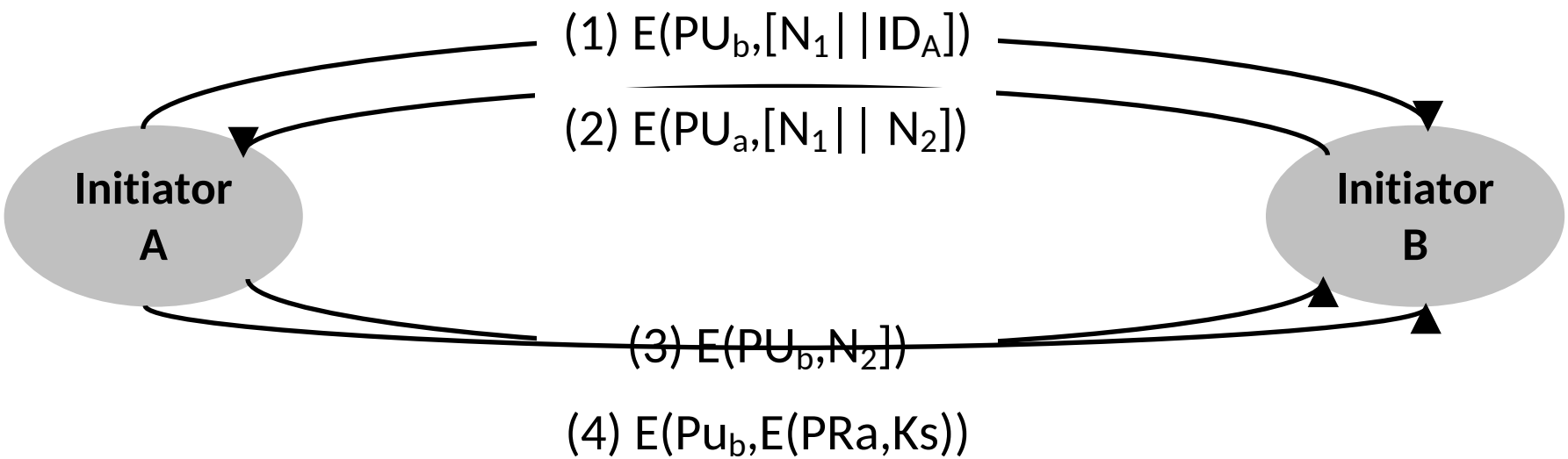
- Communication between end systems is encrypted using a temporary key, often referred to as a **session key**.
- Session keys are transmitted in encrypted form, using a **master key** that is shared by the key distribution center and an end system or user



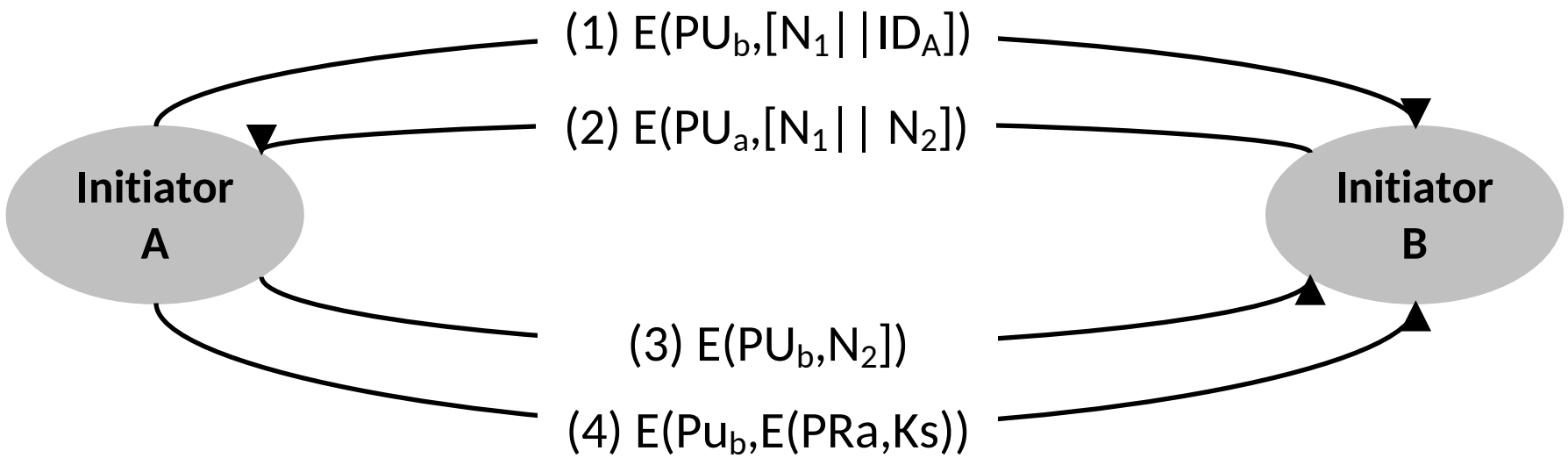
Simple Secret Key Distribution



1. **A** generates a public/private key pair **{PUa, PRa}** and transmits a message to **B** consisting of **PUa** and an identifier of **A**, **ID_A**.
2. **B** generates a secret key, **Ks**, and transmits it to **A**, encrypted with **A**'s public key.
3. **A** computes **D(PRa, E(PUa, Ks))** to recover the secret key. Because only **A** can decrypt the message, only **A** and **B** will know the identity of **Ks**.
4. **A** discards **PUa** and **PRa** and **B** discards **PUa**.



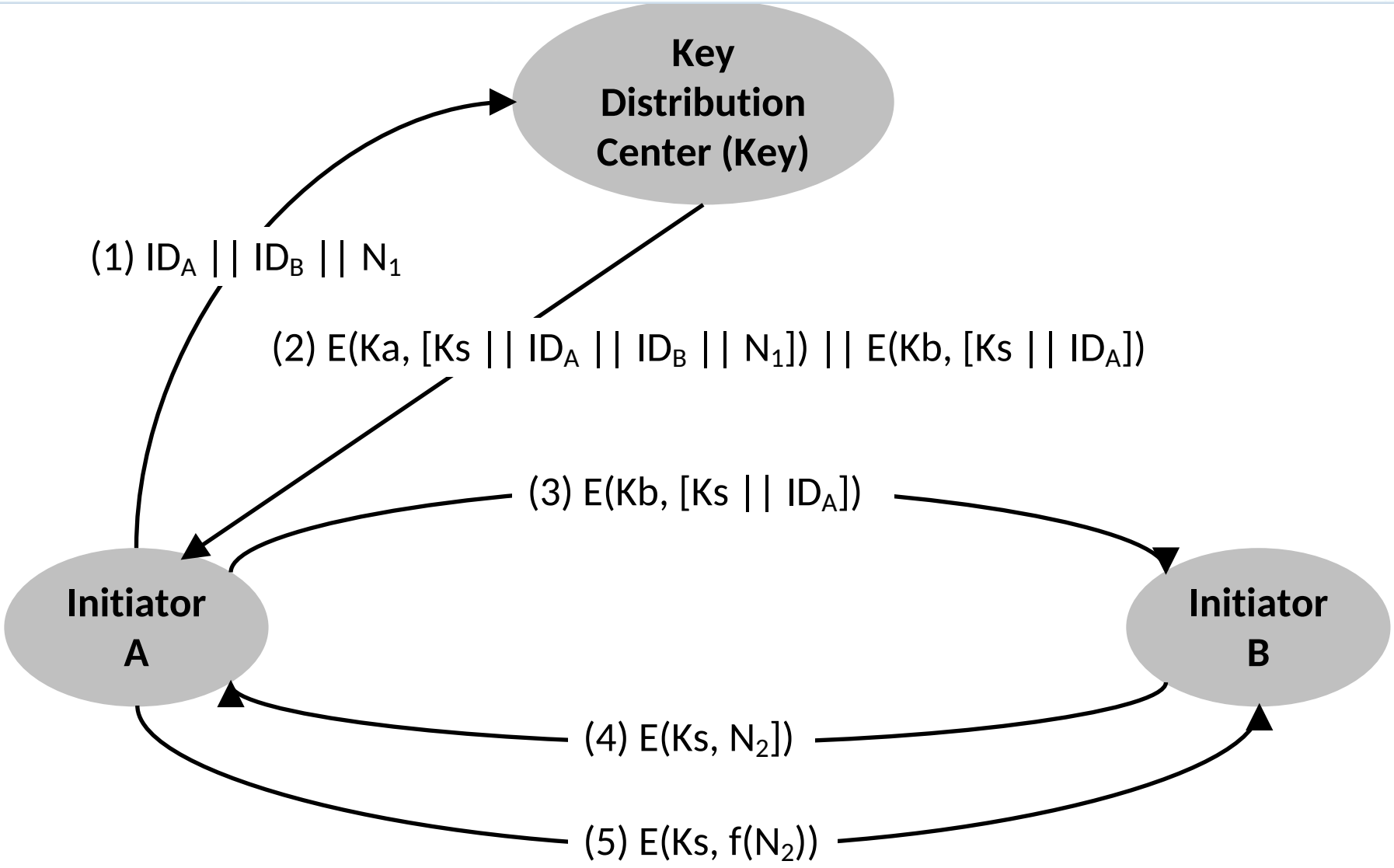
1. **A** uses **B**'s public key to encrypt a message to **B** containing an identifier of **A** (I_A) and a nonce (N_1), which is used to identify this transaction uniquely.
2. **B** sends a message to **A** encrypted with **PUa** and containing **A**'s (N_1) as well as a new nonce generated by **B** (N_2). Because only **B** could have decrypted message (1), the presence of N_1 in message (2) assures **A** that the correspondent is **B**.



3. **A** returns **N_2**, encrypted using **B**'s public key, to assure **B** that its correspondent is **A**.
4. **A** selects a secret key **Ks** and sends **M = E(PUb, E(PRa, Ks))** to **B**. Encryption with **B**'s public key ensures that only **B** can read it; encryption with **A**'s private key ensures that only **A** could have sent it.
5. **B** computes **D(PUa, D(PRb, M))** to recover the secret key.

Symmetric key distribution using symmetric encryption

- Two parties A and B, key distribution can be achieved in a number of ways, as follows:
  1. **A** can select a key and **physically deliver key** to **B**.
  2. Third party can select the key and physically deliver it to **A** and **B**.
  3. If **A** and **B** have previously and recently used a key, one party can transmit the new key to the other, encrypted using the old key.
  4. If **A** and **B** each has an encrypted connection to a third party **C**, **C** can deliver a key on the encrypted links to A and B.


Key Distribution Center (Key)
(1) ID_A || ID_B || N_1
(2) E(Ka, [Ks || ID_A || ID_B || N_1]) || E(Kb, [Ks || ID_A])
(3) E(Kb, [Ks || ID_A])
Initiator A
Initiator B
(4) E(Ks, N_2])
(5) E(Ks, f(N_2))

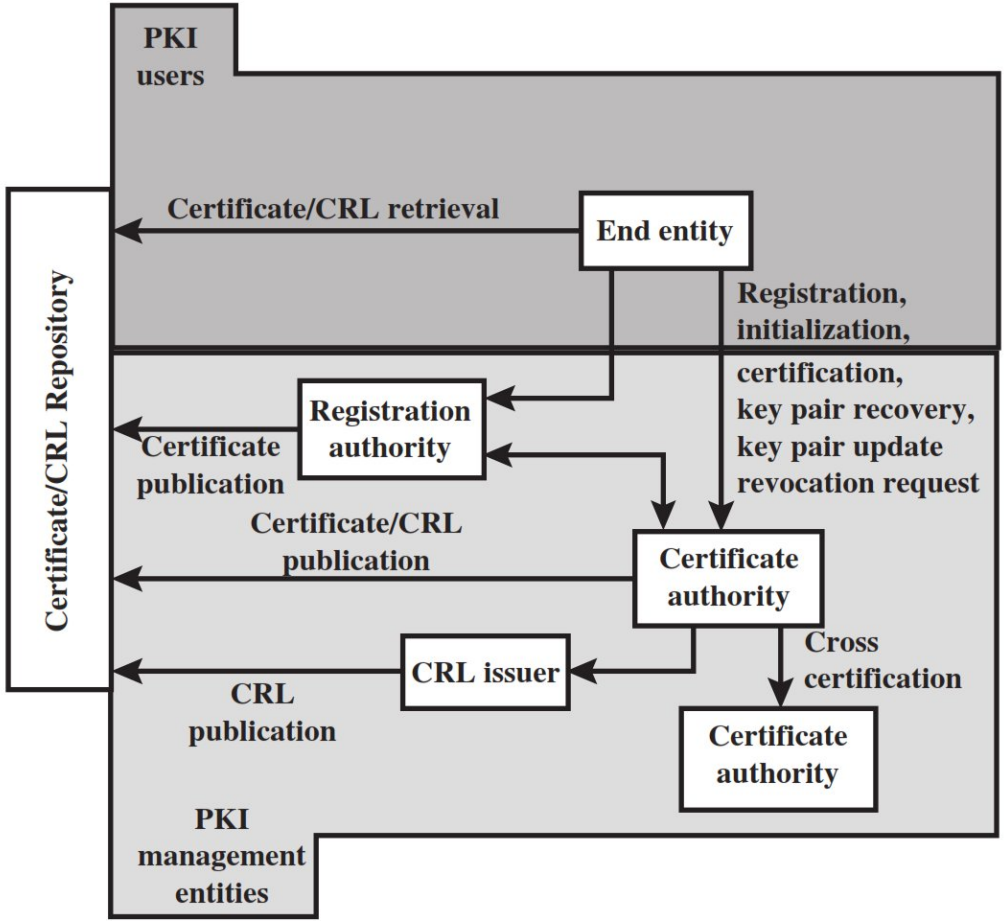
Key Distribution Scenario

1. **A** requests from the KDC a session key to protect a logical connection to **B**. The message includes the identity of **A** and **B** and a unique nonce **N1**.
2. The KDC responds with a message encrypted using **Ka** that includes a one-time session key **Ks** to be used for the session, the original request message to enable **A** to match response with appropriate request, and info for **B**
3. **A** stores the session key for use in the upcoming session and forwards to **B** the information from the KDC for **B**, namely, **E(Kb, [Ks || IDA])**.
4. At this point, a session key has been securely delivered to **A** and **B**, and they may begin their protected exchange.
5. Using the new session key for encryption B sends a nonce N_2 to **A**.
6. Also using **Ks**, **A** responds with $f(N_2)$. These steps assure **B** that the original message it received (step 3) was not a replay. Note that the actual key distribution involves only steps 1 through 3 but that steps 4 and 5, as well as 3, perform an authentication function.

Public key Infrastructure (PKI)

- A **public-key infrastructure (PKI)** is defined as the set of hardware, software, people, policies, and procedures needed to create, manage, store, distribute, and revoke digital certificates based on asymmetric cryptography.
- The principal objective for developing a PKI is to enable secure, convenient, and efficient acquisition of **public keys**.

Public key Infrastructure (PKI)



- **End entity:** A generic term used to denote end users, devices (e.g., servers, routers), or any other entity that can be identified in the subject field of a public-key certificate.
- **Certification authority (CA):** The issuer of certificates and (usually) certificate revocation lists (CRLs).
- **Registration authority (RA):** An optional component that can assume a number of administrative functions from the CA.
- **CRL issuer:** An optional component that a CA can delegate to publish CRLs.
- **Repository:** A generic term used to denote any method for storing certificates and CRLs so that they can be retrieved by end entities.

Outline

- Remote user authentication with symmetric encryption
- Remote user authentication with asymmetric encryption
- Kerberos

Mutual Authentication

1. A ---> KDC

   IDa || IDb || N1

2. KDC ---> A

   E(Ka, [Ks || IDb || N1 || E(Kb,[Ks || IDa)])

3. A ---> B

   E(Kb, [Ks || IDa])

4. B ---> A

   E(Ks, N2)

5. A ---> B

   E(Ks, f(N2))

One-way Authentication

1. A ---> KDC

   IDa || IDb || N1

2. KDC ---> A

   E(Ka, [Ks || IDb || N1 || E(Kb, [Ks || IDa])])

3. A ---> B

   E(Kb, [Ks || IDa]) || E(Ks, M)

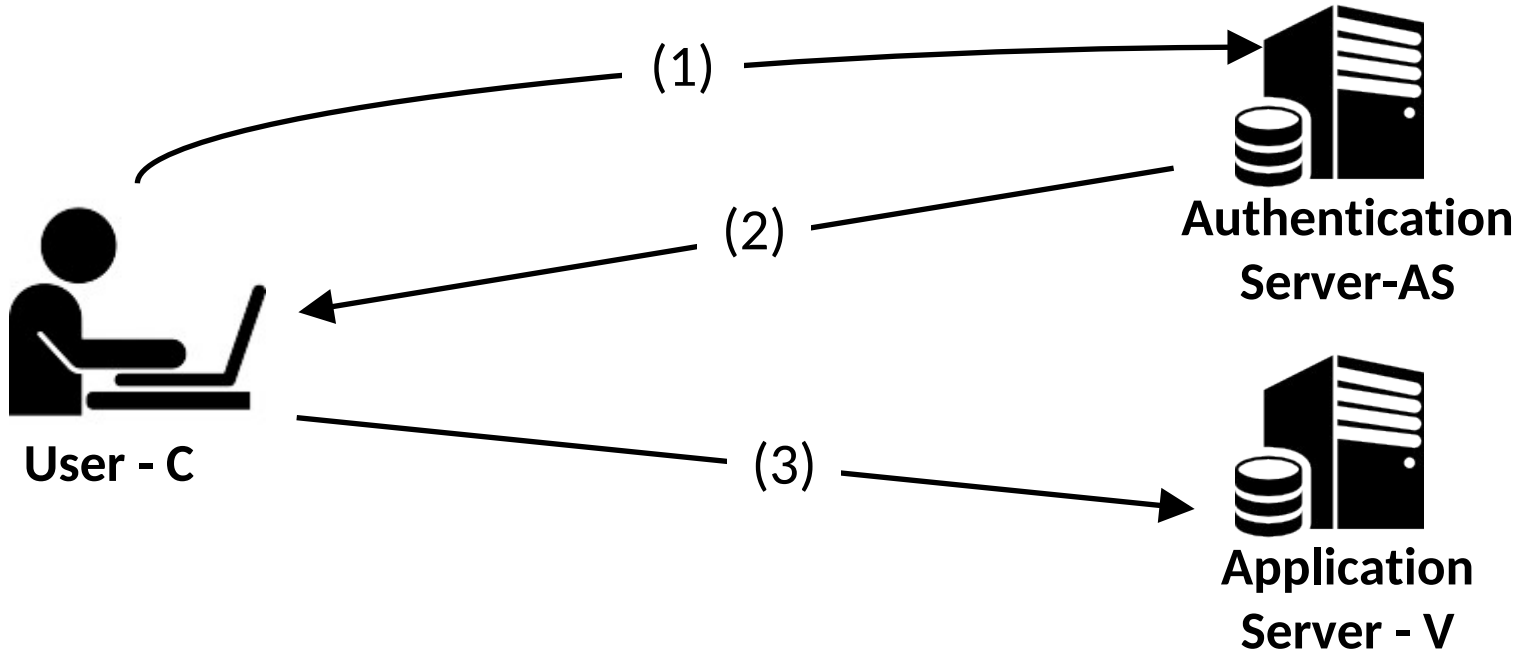
Kerberos

- **Kerberos** provides a trusted third-party authentication service that enables clients and servers to establish **authenticated communication**.
- **Kerberos** is an authentication service designed for use in a distributed environment.
- Version 4 of Kerberos makes use of **DES**, to provide the authentication service.

Kerberos – Simple Dialogue

1. C ---> AS: **IDc || Pc || IDv**
2. AS ---> C: **Ticket**
3. C ---> V: **IDc || Ticket**

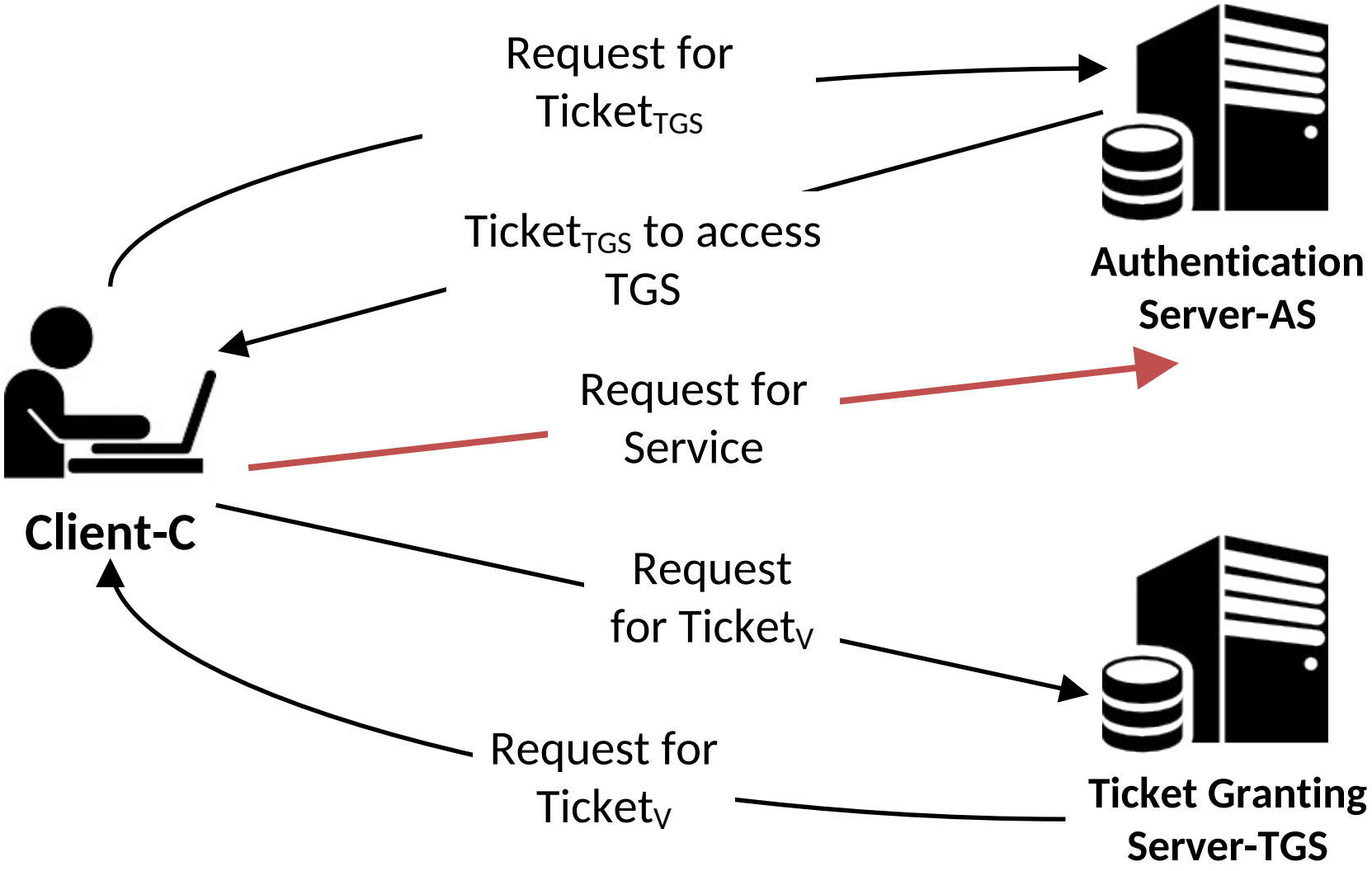
Ticket = E(Kv, [IDc, ADc, IDv])



1. C ---> AS: **IDc || Pc || IDv**
2. AS ---> C: **Ticket**
3. C ---> V: **IDc || Ticket**

Ticket = E(Kv, [IDc, ADc, IDv])

- Where,
  - C = Client
  - AS = Authentication Server
  - V = Server
  - IDc = Identification of user C
  - Idv = Identification of V
  - Pc = Password on User C
  - Adc = Network Address of C
  - Kv = Secret key shared by AS and V


Request for Ticket$_{TGS}$
Ticket$_{TGS}$ to access TGS
Authentication Server-AS
Request for Service
Client-C
Request for Ticket$_V$
Request for Ticket$_V$
Ticket Granting Server-TGS

- **Once per user logon session**

1. C ---> AS: **IDc || IDTGS**
2. AS ---> C: **E(Kc, $Ticket_{TGS}$)**

- **Once per type of service**

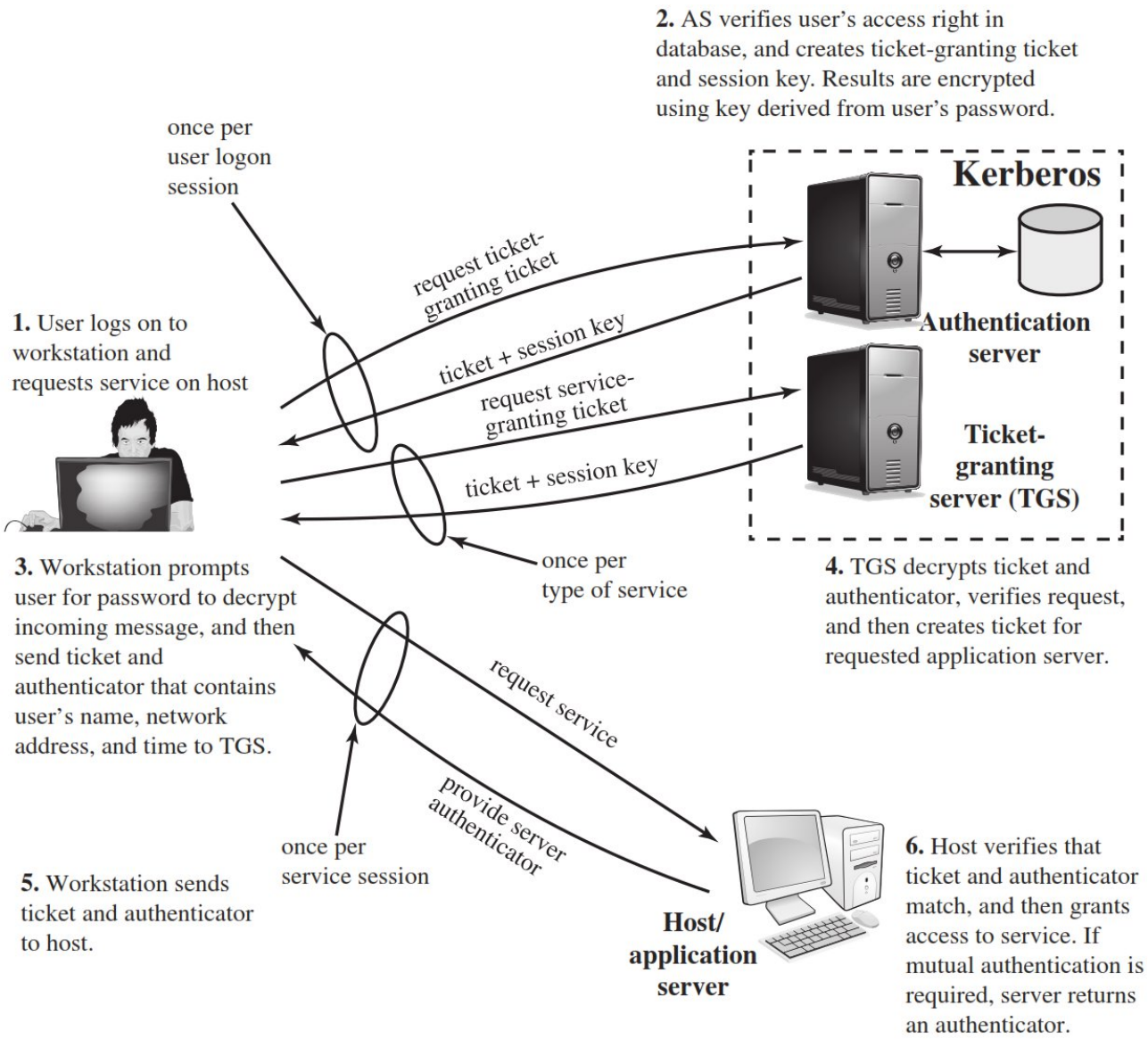
1. C ---> TGS: **IDc || IDv || $Ticket_{TGS}$**
2. TGS ---> C: **$Ticket_V$**

- **Once per service session**

1. C ---> V: **IDc || $Ticket_V$**

$Ticket_{TGS}$ = E(K_{TGS}, [IDc, ADc, Id_{TGS}, TS1, Lifetime1])

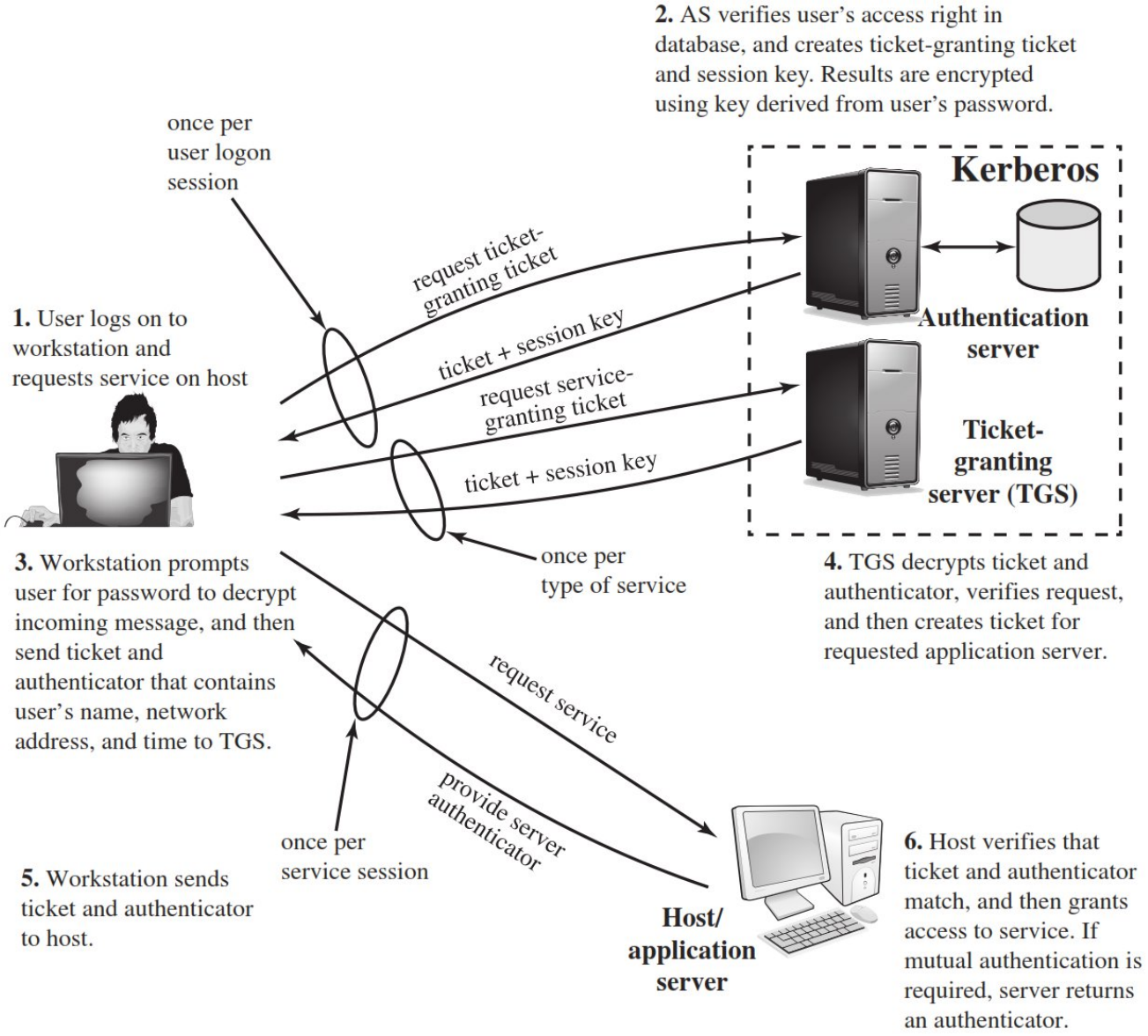
$Ticket_V$ = E(K_V, [IDc, ADc, Id_V, TS2, Lifetime2])


2. AS verifies user's access right in database, and creates ticket-granting ticket and session key. Results are encrypted using key derived from user's password.
once per user logon session
Kerberos
request ticket-granting ticket
ticket + session key
request service-granting ticket
ticket + session key
Authentication server
Ticket-granting server (TGS)
1. User logs on to workstation and requests service on host
once per type of service
3. Workstation prompts user for password to decrypt incoming message, and then send ticket and authenticator that contains user's name, network address, and time to TGS.
4. TGS decrypts ticket and authenticator, verifies request, and then creates ticket for requested application server.
request service
provide server authenticator
once per service session
5. Workstation sends ticket and authenticator to host.
Host/ application server
6. Host verifies that ticket and authenticator match, and then grants access to service. If mutual authentication is required, server returns an authenticator.

Kerberos Version 4 – Step-1

- The client sends a *plaintext* request to the **AS** asking for a ticket it can use to talk to the **TGS**.
- Request:
  - Login name
  - **TGS** name
- Since this request contains only well-known names, it does not need to be sealed.

C--->AS : IDc||ID_{tgs}||TS_1

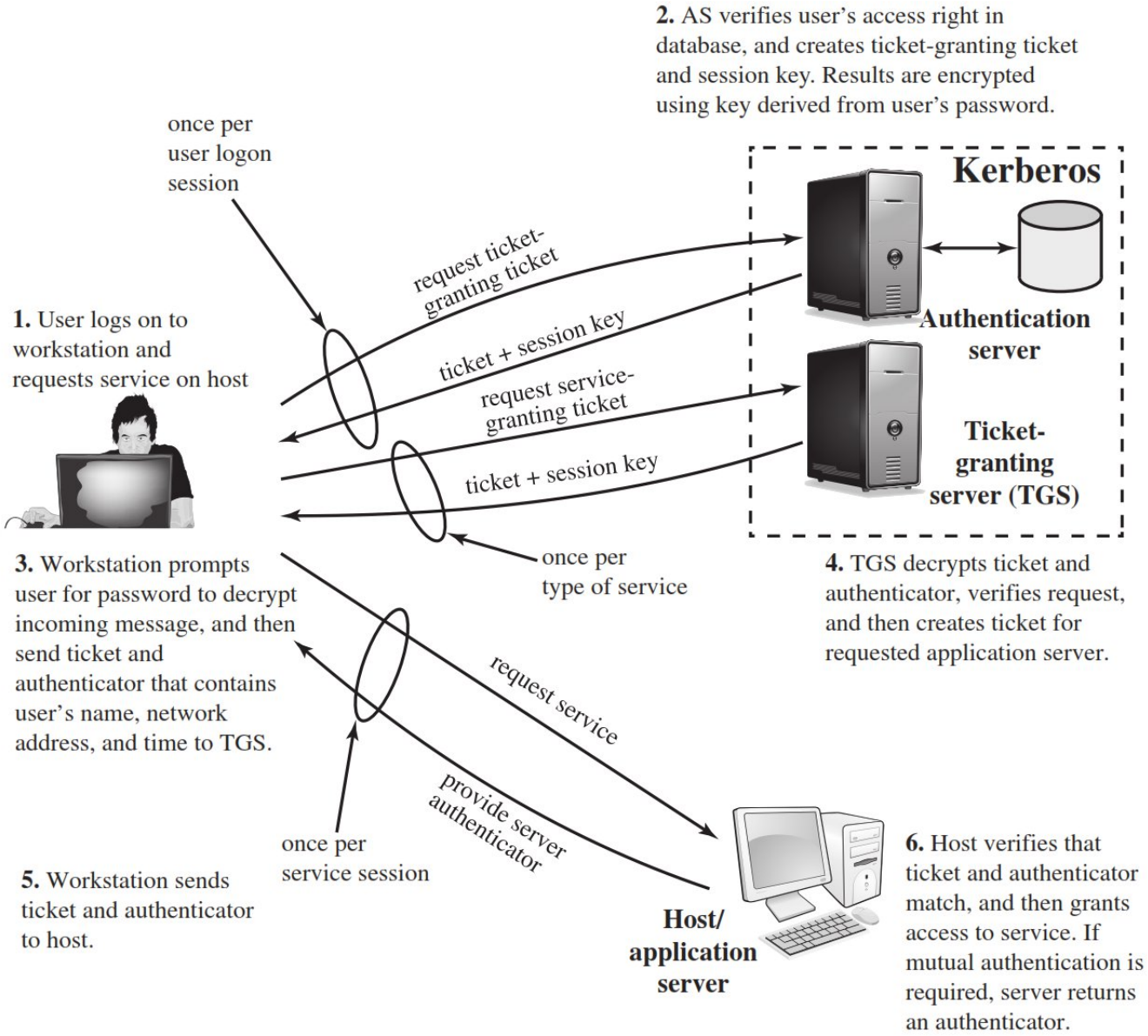

2. AS verifies user's access right in database, and creates ticket-granting ticket and session key. Results are encrypted using key derived from user's password.
once per user logon session
Kerberos
request ticket-granting ticket
ticket + session key
request service-granting ticket
ticket + session key
Authentication server
Ticket-granting server (TGS)
1. User logs on to workstation and requests service on host
once per type of service
3. Workstation prompts user for password to decrypt incoming message, and then send ticket and authenticator that contains user's name, network address, and time to TGS.
4. TGS decrypts ticket and authenticator, verifies request, and then creates ticket for requested application server.
request service
provide server authenticator
once per service session
5. Workstation sends ticket and authenticator to host.
Host/ application server
6. Host verifies that ticket and authenticator match, and then grants access to service. If mutual authentication is required, server returns an authenticator.

Kerberos Version 4 – Step-2

- The **AS** finds the keys corresponding to the login name and the **TGS** name.
- The **AS** creates a ticket:
  - Login name
  - **TGS** name
  - Client network address
  - **TGS** session key
- The **AS** seals the ticket with the **TGS** secret key.

$$AS \text{--->} C: E(Kc, [K_{c,tgs} || ID_{tgs} || TS_2 || Lifetime_2 || Ticket_{tgs}])$$

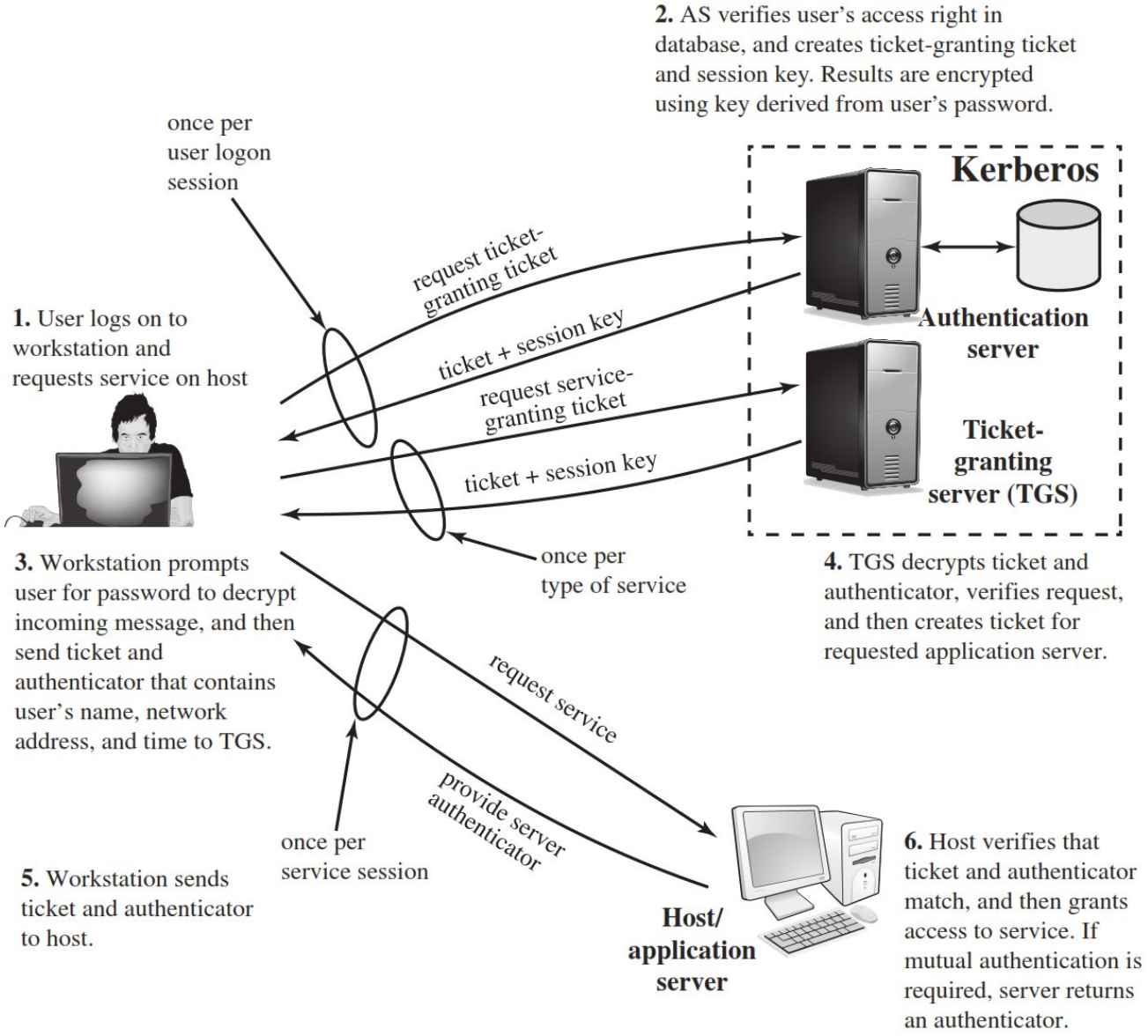
$$Ticket_{tgs} = E(K_{tgs}, [K_{c,tgs} || IDc || ADc || ID_{tgs} || TS_2 || Lifetime_2])$$


2. AS verifies user's access right in database, and creates ticket-granting ticket and session key. Results are encrypted using key derived from user's password.
once per user logon session
Kerberos
request ticket-granting ticket
ticket + session key
request service-granting ticket
ticket + session key
Authentication server
Ticket-granting server (TGS)
1. User logs on to workstation and requests service on host
once per type of service
3. Workstation prompts user for password to decrypt incoming message, and then send ticket and authenticator that contains user's name, network address, and time to TGS.
4. TGS decrypts ticket and authenticator, verifies request, and then creates ticket for requested application server.
request service
provide server authenticator
once per service session
5. Workstation sends ticket and authenticator to host.
Host/ application server
6. Host verifies that ticket and authenticator match, and then grants access to service. If mutual authentication is required, server returns an authenticator.

Kerberos Version 4 – Step-3

- The client decrypts the message using the user's password as the secret key.
- The client now has a session key and ticket that can be used to contact the **TGS**.
- The client cannot see inside the ticket, since the client does not know the **TGS** secret key.
- When a client wants to start using a server (service), the client must first obtain a ticket.
- The client composes a request to send to the **TGS**

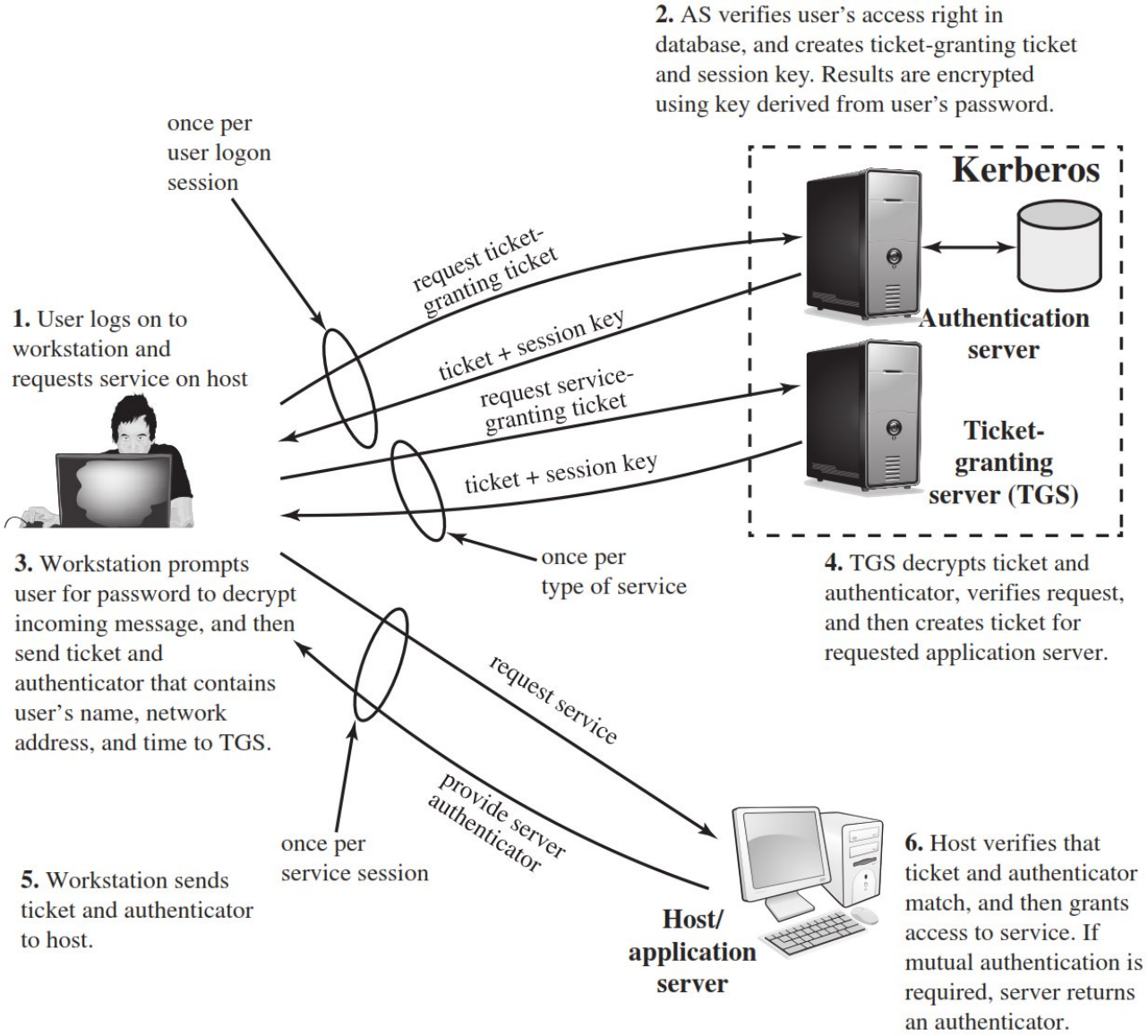
C--->TGS: Idv||$Ticket_{tgs}$||$Authenticator_c$


2. AS verifies user's access right in database, and creates ticket-granting ticket and session key. Results are encrypted using key derived from user's password.
once per user logon session
Kerberos
request ticket-granting ticket
ticket + session key
request service-granting ticket
ticket + session key
Authentication server
Ticket-granting server (TGS)
1. User logs on to workstation and requests service on host
once per type of service
3. Workstation prompts user for password to decrypt incoming message, and then send ticket and authenticator that contains user's name, network address, and time to TGS.
4. TGS decrypts ticket and authenticator, verifies request, and then creates ticket for requested application server.
request service
provide server authenticator
once per service session
5. Workstation sends ticket and authenticator to host.
Host/ application server
6. Host verifies that ticket and authenticator match, and then grants access to service. If mutual authentication is required, server returns an authenticator.

Kerberos Version 4 – Step-4

- The **TGS** decrypts the ticket using it's secret key. Inside is the TGS session key.
- The **TGS** decrypts the authenticator using the session key.
- The **TGS** check to make sure login names, client addresses and **TGS** server name are all ok.
- **TGS** makes sure the authenticator is recent.
- Builds a ticket for the client and requested server. The ticket is sealed with the server key.
- Creates a session key
- Seals the entire message with the tgs session key and sends it to the client.

TGS--->C: $E(K_{c,tgs}, [K_{c,v}||IDv||TS_4||Ticket_v])$

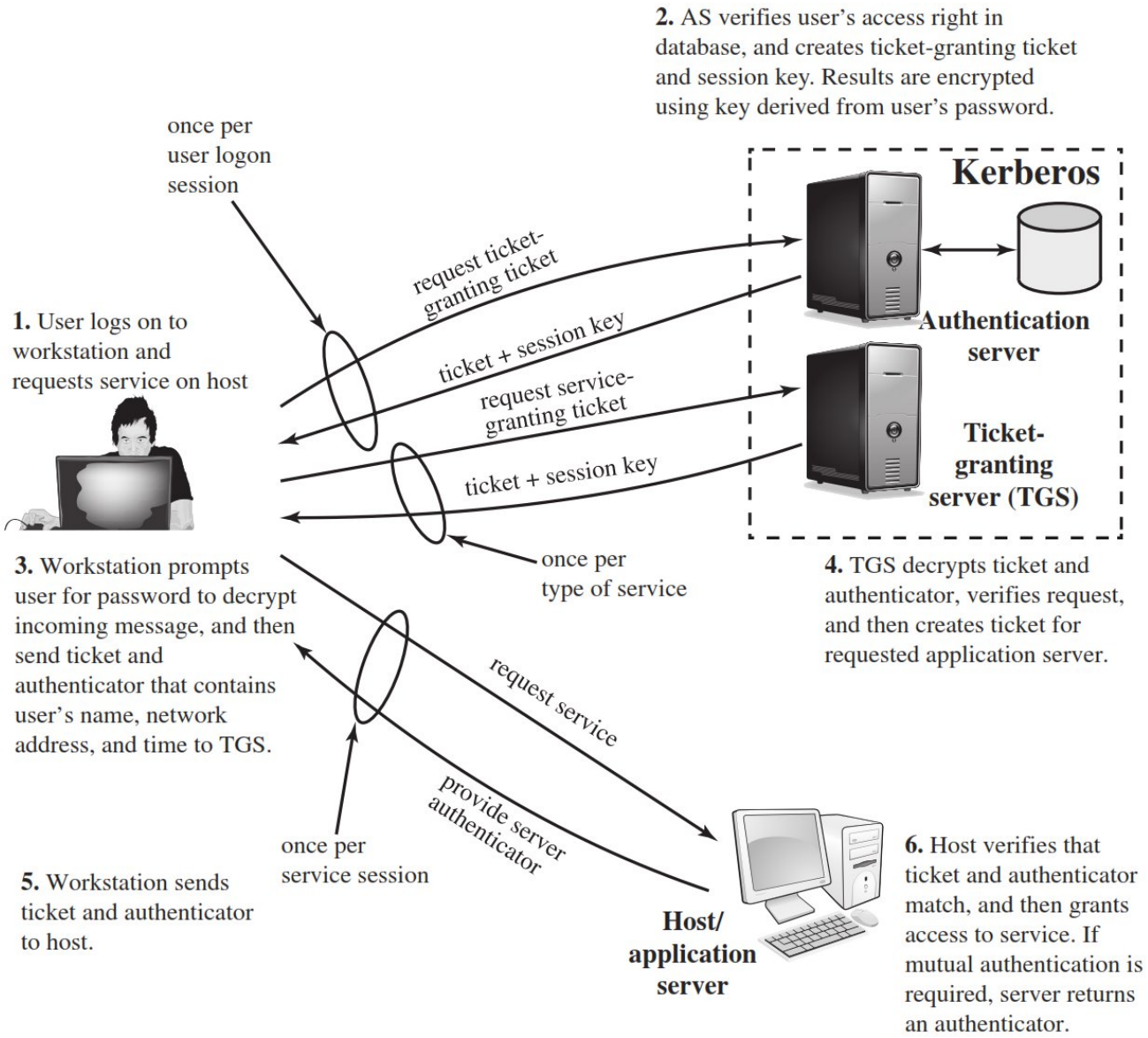

2. AS verifies user's access right in database, and creates ticket-granting ticket and session key. Results are encrypted using key derived from user's password.
once per user logon session
Kerberos
request ticket-granting ticket
ticket + session key
request service-granting ticket
ticket + session key
Authentication server
Ticket-granting server (TGS)
1. User logs on to workstation and requests service on host
once per type of service
3. Workstation prompts user for password to decrypt incoming message, and then send ticket and authenticator that contains user's name, network address, and time to TGS.
4. TGS decrypts ticket and authenticator, verifies request, and then creates ticket for requested application server.
request service
provide server authenticator
once per service session
5. Workstation sends ticket and authenticator to host.
Host/ application server
6. Host verifies that ticket and authenticator match, and then grants access to service. If mutual authentication is required, server returns an authenticator.

Kerberos Version 4 – Step-4

$$\mathbf{Ticket_{tgs}=E(K_{tgs}, [K_{c,tgs}||IDc||ADc||ID_{tgs}||TS_2||Lifetime_2])}$$

$$\mathbf{Ticket_{V}=E(K_{V}, [K_{c,V}||IDc||ADc||ID_{V}||TS_4||Lifetime_4])}$$

$$\mathbf{Authenticator_{C}=E(K_{c,tgs}, [IDc||ADc||TS_3])}$$


2. AS verifies user's access right in database, and creates ticket-granting ticket and session key. Results are encrypted using key derived from user's password.
once per user logon session
Kerberos
request ticket-granting ticket
ticket + session key
Authentication server
1. User logs on to workstation and requests service on host
request service-granting ticket
ticket + session key
Ticket-granting server (TGS)
once per type of service
3. Workstation prompts user for password to decrypt incoming message, and then send ticket and authenticator that contains user's name, network address, and time to TGS.
4. TGS decrypts ticket and authenticator, verifies request, and then creates ticket for requested application server.
request service
provide server authenticator
once per service session
5. Workstation sends ticket and authenticator to host.
Host/ application server
6. Host verifies that ticket and authenticator match, and then grants access to service. If mutual authentication is required, server returns an authenticator.

Kerberos Version 4 – Step-5, 6

- The client now decrypts the **TGS** response using the TGS session key.
- The client now has a session key for use with the new server, and a ticket to use with that server.
- The client can contact the new server using the same format used to access the **TGS**.

$$\mathbf{Ticket_V = E(K_V, [K_{c,V} || IDc || ADc || ID_V || TS_4 || Lifetime_4])}$$

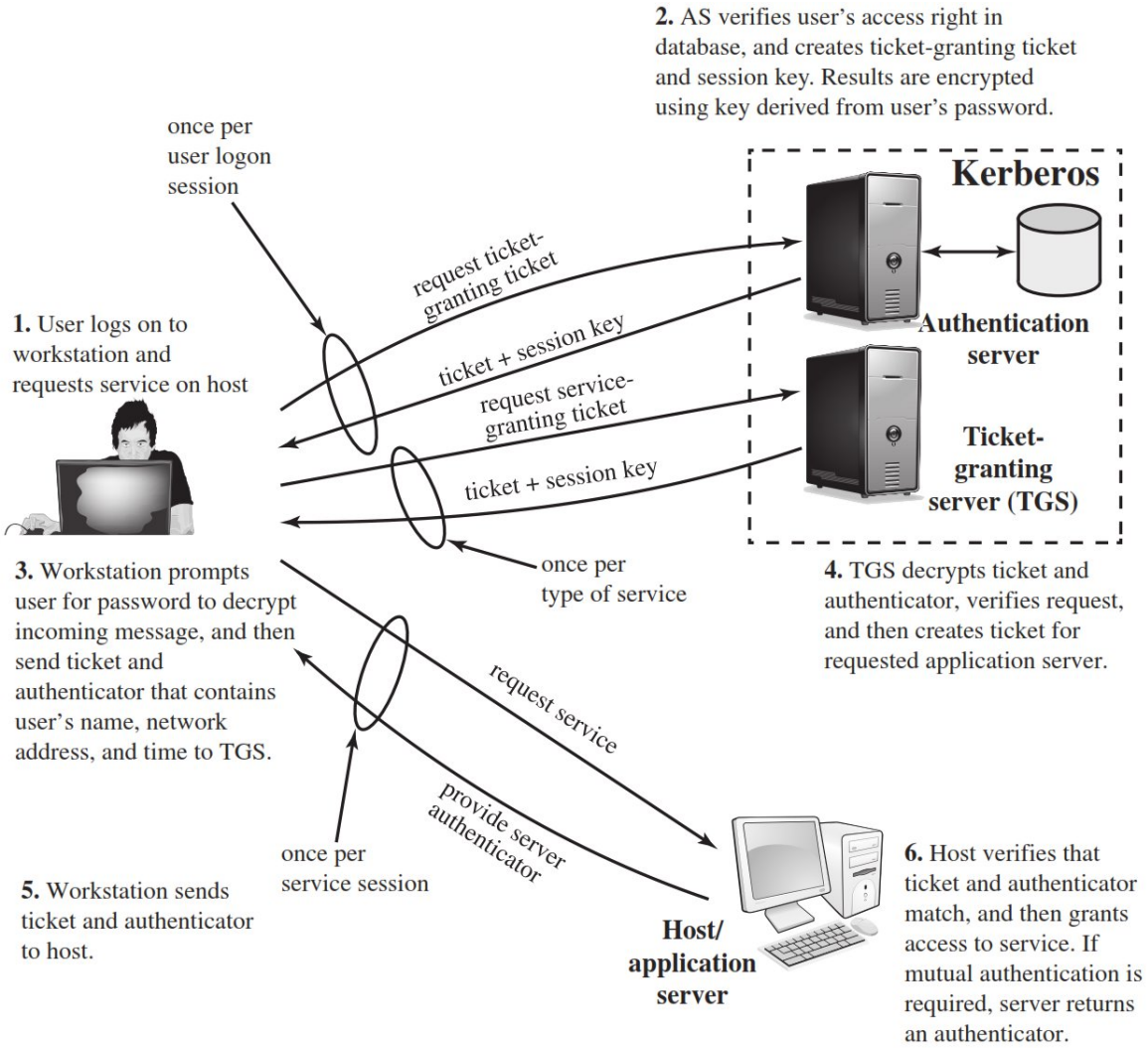
$$\mathbf{Authenticator_C = E(K_{c,v}, [IDc || ADc || TS_5])}$$

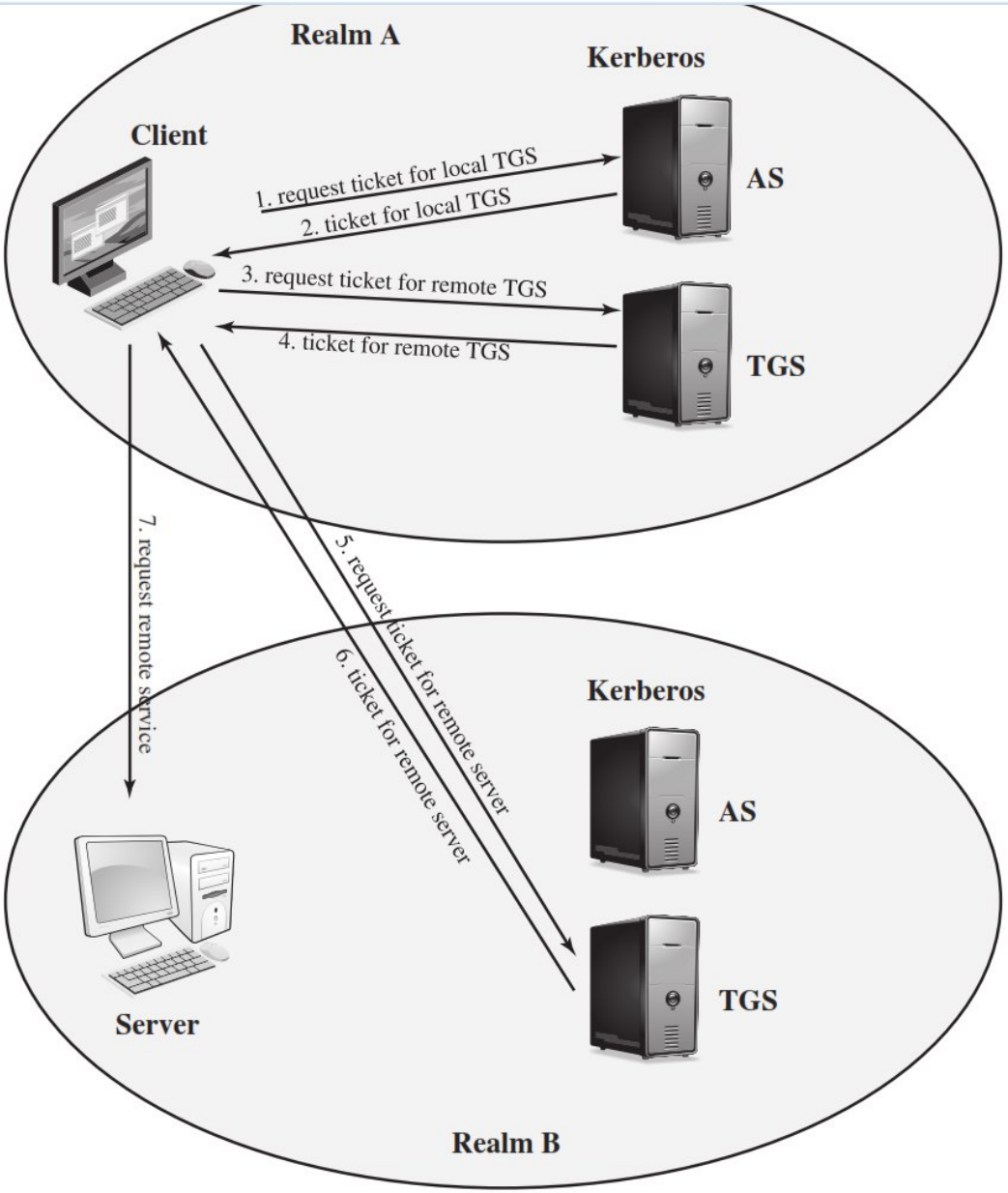
Ticket

- Each request for a service requires a ticket.
- A ticket provides a single client with access to a single server.
- Tickets are dispensed by the "ticket granting server" (**TGS**), which has knowledge of all the encryption keys.
- Tickets are meaningless to clients, they simply use them to gain access to servers.
- The **TGS** seals (encrypts) each ticket with the secret encryption key of the server.
- Sealed tickets can be sent safely over a network - only the server can make sense out of it.
- Each ticket has a limited lifetime (a few hours).

Ticket Contents

- Client name (user login name)
- Server name
- Client host network address
- Session key for client/server
- Ticket lifetime
- Creation timestamp


2. AS verifies user's access right in database, and creates ticket-granting ticket and session key. Results are encrypted using key derived from user's password.
once per user logon session
Kerberos
request ticket-granting ticket
ticket + session key
Authentication server
1. User logs on to workstation and requests service on host
request service-granting ticket
ticket + session key
Ticket-granting server (TGS)
once per type of service
3. Workstation prompts user for password to decrypt incoming message, and then send ticket and authenticator that contains user's name, network address, and time to TGS.
4. TGS decrypts ticket and authenticator, verifies request, and then creates ticket for requested application server.
request service
provide server authenticator
once per service session
5. Workstation sends ticket and authenticator to host.
Host/ application server
6. Host verifies that ticket and authenticator match, and then grants access to service. If mutual authentication is required, server returns an authenticator.



Kerberos Realm

Remote user authentication with Asymmetric encryption

Mutual Authentication

1. $A \rightarrow AS: \quad ID_A \| ID_B$
2. $AS \rightarrow A: \quad E(PR_{as}, [ID_A \| PU_a \| T]) \| E(PR_{as}, [ID_B \| PU_b \| T])$
3. $A \rightarrow B: \quad E(PR_{as}, [ID_A \| PU_a \| T]) \| E(PR_{as}, [ID_B \| PU_b \| T]) \| E(PU_b, E(PR_a, [K_s \| T]))$

1. $A \rightarrow KDC: \quad ID_A \| ID_B$
2. $KDC \rightarrow A: \quad E(PR_{auth}, [ID_B \| PU_b])$
3. $A \rightarrow B: \quad E(PU_b, [N_a \| ID_A])$
4. $B \rightarrow KDC: \quad ID_A \| ID_B \| E(PU_{auth}, N_a)$
5. $KDC \rightarrow B: \quad E(PR_{auth}, [ID_A \| PU_a]) \| E(PU_b, E(PR_{auth}, [N_a \| K_s \| ID_B]))$
6. $B \rightarrow A: \quad E(PU_a, [E(PR_{auth}, [(N_a \| K_s \| ID_B)]) \| N_b])$
7. $A \rightarrow B: \quad E(K_s, N_b)$